WEBX Module 3

Explain Multilevel Inheritance in TypeScript with a suitable example.

Multilevel Inheritance refers to a mechanism where a class inherits from another class, which in turn inherits from another class. This forms a chain of inheritance (Grandparent → Parent → Child).

Example:

```
// Grandparent class
class Animal {
    eat(): void {
        console.log("This animal eats food.");
    }
}

// Parent class inherits Animal
class Mammal extends Animal {
    breathe(): void {
        console.log("This mammal breathes air.");
    }
}

// Child class inherits Mammal
class Dog extends Mammal {
    bark(): void {
        console.log("The dog barks.");
    }
}

// Usage
const myDog = new Dog();
myDog.eat();     // Inherited from Animal
myDog.breathe(); // Inherited from Mammal
myDog.bark();    // Own method
```

Differentiate between JavaScript and TypeScript

Feature	JavaScript	TypeScript
Type System	Dynamically typed	Statically typed (optional)
Compilation	Interpreted directly	Compiled to JavaScript
Tooling	Limited autocompletion	Excellent IDE support & IntelliSense
Features	ES6+ features	All ES6+ features + Generics, Enums, Decorators
Error Checking	Runtime errors	Compile-time error checking
File Extension	`.js`	`.ts`
Learning Curve	Easier for beginners	Steeper due to types & concepts

---

State the features of TypeScript

1. Static Typing – Variables, parameters, and return types can be explicitly typed.
2. Object-Oriented – Supports classes, interfaces, inheritance, modules.
3. Compile-time Checking – Catches errors during development, not runtime.
4. ES6+ Support – Supports modern JavaScript features (async/await, arrow functions, destructuring).
5. Generics – Create reusable components with type safety.
6. Decorators – Metadata and annotations (used in Angular).
7. Namespace & Modules – Better code organization.
8. Type Inference – TypeScript automatically infers types when not specified.

Explain an arrow function in TypeScript

Arrow functions provide a concise syntax and lexically bind the `this` value (no own `this` context).

Syntax: `(parameters) => expression` or `(parameters) => { block }`

Examples:

```
// Basic arrow function
const add = (a: number, b: number): number => a + b;
```

```
// Without parameters
const greet = (): string => "Hello";

// Single parameter (parentheses optional)
const square = (x: number): number => x * x;

// Lexical this binding
class Timer {
    count: number = 0;
    start() {
        setInterval(() => {
            this.count++; // 'this' refers to Timer instance
            console.log(this.count);
        }, 1000);
    }
}
```

Differentiate Var v/s Let. In addition, explain function overloading with suitable examples.

Difference between `var` and `let`:

Feature	`var`	`let`
Scope	Function-scoped	Block-scoped `{}`
Hoisting	Hoisted (initialized as `undefined`)	Hoisted but not initialized (TDZ)
Redeclaration	Allowed in same scope	Not allowed
Global property	Becomes property of `window`	Does not become global property

Example:

```
// var example
function varExample() {
    var x = 1;
    if (true) {
        var x = 2; // Same variable!
    }
    console.log(x); // 2
}

// let example
```

```
function letExample() {
    let y = 1;
    if (true) {
        let y = 2; // Different variable (block-scoped)
    }
    console.log(y); // 1
}
```

Function Overloading in TypeScript

TypeScript supports overloading by defining multiple function signatures (but single implementation).

Example:

```
// Overload signatures
function combine(a: string, b: string): string;
function combine(a: number, b: number): number;

// Implementation
function combine(a: any, b: any): any {
    if (typeof a === "string" && typeof b === "string") {
        return a + b;
    } else if (typeof a === "number" && typeof b === "number") {
        return a + b;
    }
}

// Usage
let result1 = combine("Hello", " World"); // string
let result2 = combine(10, 20);            // number
```

Discuss definite and indefinite loops with suitable examples in TypeScript

Definite Loops – known number of iterations:

```
// for loop
for (let i: number = 1; i <= 5; i++) {
    console.log(`Iteration: ${i}`);
}
```

```
// for...of (iterates over values)
let fruits: string[] = ["Apple", "Banana", "Mango"];
for (let fruit of fruits) {
    console.log(fruit);
}

// for...in (iterates over keys/properties)
for (let index in fruits) {
    console.log(index); // "0", "1", "2"
}
```

Indefinite Loops – unknown number of iterations:

```
// while loop
let count: number = 0;
while (count < 3) {
    console.log(`Count: ${count}`);
    count++;
}

// do...while (executes at least once)
let input: number = 0;
do {
    console.log(`Input is ${input}`);
    input++;
} while (input < 0); // Executes once even though condition is false

// Infinite loop (with break condition)
let attempts: number = 0;
while (true) {
    attempts++;
    if (attempts === 5) break;
    console.log(`Attempt ${attempts}`);
}
```

Explain Modules in TypeScript with example

Modules organize code into separate files, each with its own scope. TypeScript supports ES Modules.

mathUtils.ts (exporting):

```ts
// Named exports
export const PI = 3.14159;
export function add(x: number, y: number): number {
    return x + y;
}

// Default export
export default function multiply(x: number, y: number): number {
    return x * y;
}
```

app.ts (importing):

```ts
import multiply, { add, PI } from "./mathUtils.js";

console.log(add(5, 3));      // 8
console.log(multiply(4, 2)); // 8 (default import)
console.log(PI);              // 3.14159

// Import all as namespace
import * as MathUtils from "./mathUtils.js";
console.log(MathUtils.add(1, 2));
```

Explain how internal and external Modules are used in TypeScript with suitable example

Internal Modules (Namespaces) – Deprecated, but still seen in old code
Used to organize code within a single file/global scope.

```ts
// Internal module (namespace)
namespace Validation {
    export interface StringValidator {
        isValid(s: string): boolean;
    }

    export class EmailValidator implements StringValidator {
        isValid(s: string): boolean {
            return s.includes("@");
        }
    }
}
```

```
// Usage
let validator = new Validation.EmailValidator();
console.log(validator.isValid("test@example.com")); // true
```

External Modules (Modern ES Modules)

mathUtils.ts (exporting):

```
// Named exports
export const PI = 3.14159;
export function add(x: number, y: number): number {
    return x + y;
}

// Default export
export default function multiply(x: number, y: number): number {
    return x * y;
}
```

app.ts (importing):

```
import multiply, { add, PI } from "./mathUtils.js";

console.log(add(5, 3));      // 8
console.log(multiply(4, 2)); // 8 (default import)
console.log(PI);              // 3.14159

// Import all as namespace
import * as MathUtils from "./mathUtils.js";
console.log(MathUtils.add(1, 2));
```

Note: Modern TypeScript uses ES Modules. Namespaces (internal modules) are discouraged for new code.

Explain Arithmetic operators with suitable example

Arithmetic operators are used to perform mathematical operations.

Operator	Name	Example	Result
`+`	Addition	`5 + 3`	`8`

Operator	Name	Example	Result
`-`	Subtraction	`5 - 3`	`2`
`*`	Multiplication	`5 * 3`	`15`
`/`	Division	`10 / 3`	`3.333`
`%`	Modulus (Remainder)	`10 % 3`	`1`
`**`	Exponentiation (ES7)	`2  3`	`8`
`++`	Increment	`let x=5; x++`	`6`
`--`	Decrement	`let x=5; x--`	`4`

Example:

```
let a: number = 15;
let b: number = 4;

console.log(`Addition: ${a + b}`);          // 19
console.log(`Subtraction: ${a - b}`);       // 11
console.log(`Multiplication: ${a * b}`);    // 60
console.log(`Division: ${a / b}`);          // 3.75
console.log(`Modulus: ${a % b}`);           // 3
console.log(`Exponent: ${a ** b}`);         // 15^4 = 50625

// Increment/Decrement
let counter: number = 10;
counter++;
console.log(`After increment: ${counter}`); // 11
counter--;
console.log(`After decrement: ${counter}`); // 10

// Compound assignment with arithmetic
let total: number = 5;
total += 3;  // total = total + 3
console.log(total); // 8
```